



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U04_A04_CPE Caso práctico extendido – Prof. Emilio
Ramírez**

Autor: Segura Abad, Mario

Fecha: 5 de Febrero, 2026

INDICE

INTRODUCCIÓN..... 3

ENUNCIADO 4

GESTOR DE GASTOS: CONEXIÓN CON API DE SERVIDOR4

DESARROLLO DE LA PRACTICA 5

DESCRIPCIÓN5

- *Integrar una API NodeJS para leer, crear actualizar y borrar gastos (Caso práctico extendido U04_A04_CPE).5*
- *Adaptar la aplicación web para que no cargue los gastos de un usuario introducido en un formulario.....5*
- *Sustituir las locales por peticiones GET, POST, PUT y DELETE a la API.....5*
- *Comprender la arquitectura REST de la API y el formato JSON devuelto.....5*

CONTENIDO DEL ARCHIVO.....5

Caso práctico extendido U04_A04_CPE – Gestor de gastos conectado a API de servidor5

CÓMO VISUALIZARLO8

Requisitos:8

Pasos:.....8

OBJETIVOS DE LA PRÁCTICA.....8

- *Integrar una API real en la aplicación Gestor de gastos.8*
- *Realizar operaciones CRUD mediante peticiones HTTP.....8*
- *Analizar respuestas JSON y manejar errores.....8*
- *Documentar correctamente cada ejercicio con nombre, fecha y capturas de pantalla.8*
- *Presentar el proyecto en PDF junto con los archivos fuente y pantallazos.8*

CONCLUSIÓN..... 9

INTRODUCCIÓN

En este ejercicios deberemos realizar un “Gestor de gastos con conexión con API de servidor” en el cual realizaremos dicho caso práctico que ejemplifica los contenidos mediante los archivos adjuntos en la tarea para completar el caso práctico extendido.

ENUNCIADO

Gestor de gastos: conexión con API de servidor

Caso práctico que ejemplifica los contenidos.

Descargad los archivos de proyecto para completar el caso práctico extendido.

DESARROLLO DE LA PRACTICA

Práctica DWEK – Unidad 4 Apartado 4

Descripción

Este apartado se centra en la **conexión del Gestor de gastos con una API de servidor**, sustituyendo la lógica local por peticiones HTTP reales mediante fetch.

El objetivo principal ha sido:

- Integrar una API NodeJS para leer, crear actualizar y borrar gastos (Caso práctico extendido U04_A04_CPE).
- Adaptar la aplicación web para que no cargue los gastos de un usuario introducido en un formulario.
- Sustituir las locales por peticiones GET, POST, PUT y DELETE a la API.
- Comprender la arquitectura REST de la API y el formato JSON devuelto.

Contenido del archivo

Caso práctico extendido U04_A04_CPE – Gestor de gastos conectado a API de servidor

Objetivo:

- Modificar la aplicación Gestor de gastos para que funcione **exclusivamente** mediante peticiones HTTP a la API.
- Añadir un formulario para introducir el nombre del usuario cuyos gastos se van a gestionar.
- Cargar los gastos del usuario mediante GET <http://localhost:3000/USUARIO>.
- Añadir nuevos gastos mediante POST <http://localhost:3000/USUARIO>.
- Actualizar gastos mediante PUT <http://localhost:3000/USUARIO/gastoId>.
- Borrar gastos mediante DELETE <http://localhost:3000/USUARIO/gastoId>.

Claves de resolución:

- Descargar y descomprimir la API proporcionada.
- Instalar dependencias → `npm install`
- Arrancar la API → `npm run start`
- Probar usuarios de ejemplo:
 - <http://localhost:3000/usuario1>
 - <http://localhost:3000/usuario2>
- Crear el formulario para introducir el nombre del usuario.
- Sustituir las funciones locales del archivo `gestionPresupuesto.js` por peticiones fetch.
- Comprobar en las herramientas de desarrollador:
 - Método correcto (GET, POST, PUT, DELETE).
 - URL correcta.
 - Datos enviados en JSON.
 - Datos recibidos y su estructura.
- Recordar que los **id de los gastos los genera el servidor de forma automática**.

Este ha reforzado el uso de APIs REST, peticiones HTTP y la integración completa entre frontend y backend.

Ejemplo de ejecución:

Captura de pantalla – gestionPresupuesto.js

```
1 // Autor: Mario Segura Abad
2 // Fecha: 05/02/2026
3
4
```

Captura de pantalla – index.html

```
1 <!-- Autor: Mario Segura Abad -->
2 <!-- Fecha: 05/02/2026 -->
3
```

Captura de pantalla – Comando para ejecutar el servidor

```
○ mariossegura@MacBook-Air-de-Mario DWEC_U04_A04_CPE_E_archivos % npm run start
> api_gastos@1.0.0 start
> node index.js

Servidor escuchando en el puerto 3000
█
```

Salida por consola – Gestor de Gastos con todas sus funcionalidades



The screenshot shows a web application titled "Gestor de Gastos". Below the title is a "Bienvenido" (Welcome) message. A prompt asks the user to "Introduce tu usuario para cargar tus datos (ej: usuario1)". There is a text input field with the placeholder "Nombre de usuario..." and a blue "Entrar" (Enter) button.

Gestor de Gastos

Usuario: **usuario1**

Cambiar Usuario

Añadir Nuevo Gasto

Descripción:

Ej: Compra supermercado

Valor (€):

0.00

Guardar Gastos

Mis Gastos

Descripción del gasto 1 del usuario 1

25 €

Eliminar

Mis Gastos		
Descripción del gasto 1 del usuario 1	25 €	Eliminar
Compra Mercedes-Benz W211 E280 CDI W211 Avantgarde 2007	7990 €	Eliminar

127.0.0.1:5500 dice

¿Seguro que quieres borrar este gasto?

Cancelar

Aceptar

Cómo visualizarlo

Requisitos:

- Editor de código: en mi caso, Visual Studio Code.
- Navegador web actualizado: en mi caso, Google Chrome.
- Node.js para ejecutar la API.

Pasos:

1. Descomprimir la API y ejecutar `npm install`.
2. Arrancar la API con `npm run start`.
3. Abrir la aplicación Gestor de gastos en el navegador (con Live Server o directamente).
4. Introducir un usuario en el formulario y cargar sus gastos desde la API.
5. Probar añadir, modificar y borrar gastos observando las peticiones en la pestaña Network.
6. Revisar la consola (F12 → “Consola”) para verificar mensajes de autoría.

Objetivos de la práctica

- Integrar una API real en la aplicación Gestor de gastos.
- Realizar operaciones CRUD mediante peticiones HTTP.
- Analizar respuestas JSON y manejar errores.
- Documentar correctamente cada ejercicio con nombre, fecha y capturas de pantalla.
- Presentar el proyecto en PDF junto con los archivos fuente y pantallazos.

CONCLUSIÓN

Esta práctica nos ha permitido consolidar el uso de fetch, APIs REST y comunicación cliente-servidor.

Se ha comprobado cómo integrar un backend real con una aplicación web, cómo enviar y recibir datos en JSON y cómo manejar errores y peticiones HTTP.

La práctica, asimismo, nos ha sido súper útil para entender el funcionamiento de APIs modernas y la interacción entre frontend y backend.